LAPORAN PRAKTIKUM PEMROGRAMAN WEB

Mengubah, Menghapus, dan Menelusuri Data Produk

Panduan langkah demi langkah implementasi Pertemuan 9 dan 10 pada aplikasi POS Barokah Mart

Laravel 13 · Blade · Tailwind CSS · MySQL

Nama	DeandraPutra Yuriansyah
NPM	2410631170133
Mata kuliah	Pemrograman Web (FPW)
Studi kasus	POS Barokah Mart
Framework	Laravel 13.29
Database	MySQL / MariaDB
Tanggal	8 November 2026

Disusun oleh **DeandraPutra Yuriansyah** (NPM 2410631170133). Dokumen ini disusun berdasarkan implementasi nyata pada proyek **C:\xampp\htdocs\FPW\FPW_Test1**. Seluruh potongan kode di dalamnya diambil langsung dari berkas proyek yang sudah diuji.

DAFTAR ISI

Isi Dokumen

1 Pendahuluan: tujuan, prasyarat, dan hasil akhir

2 Pertemuan 9 — Mengubah dan Menghapus Data Produk

3 Pertemuan 10 — Search, Filter Kategori, dan Pagination

4 Kesimpulan dan checklist pengujian

5 Pemecahan masalah (troubleshooting)

6 Daftar berkas yang diubah dan dibuat

7 Referensi

CARA MEMBACA DOKUMEN

Setiap bab disusun mengikuti alur praktikum: konsep singkat, potongan kode yang dipakai, penjelasan baris penting, lalu langkah pengujian. Kode yang ditampilkan adalah kode final yang ada di proyek, bukan pseudocode.

BAB 1

Pendahuluan

Konteks, prasyarat, dan hasil yang akan dicapai

1.1 Tujuan pembelajaran

- Mengimplementasikan fitur ubah (*update*) data produk melalui **Route Model Binding**.
- Membuat form edit yang aman dengan `@csrf`, `@method('PUT')`, `old()`, dan pesan validasi per field.
- Mengimplementasikan fitur hapus (*delete*) data disertai konfirmasi.
- Memahami pertimbangan *soft delete* pada data yang memiliki relasi historis.
- Memahami query parameter pada URL dan cara membacanya di server.
- Menerapkan pencarian produk berdasarkan nama/kode, filter kategori, dan pagination yang mempertahankan parameter pencarian.

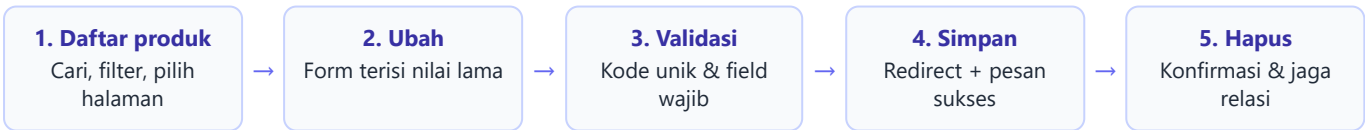
1.2 Prasyarat

- Pertemuan 1–8 sudah selesai: autentikasi, otorisasi berbasis peran, layout Blade, data master kategori, serta form tambah produk.
- PHP 8.3 atau lebih baru, Composer, Node.js, dan MySQL/MariaDB sudah berjalan.
- Aset frontend sudah dibangun (`npm run build`) atau server Vite sedang berjalan.
- Database `laravel` berisi tabel `categories`, `products`, dan `users` dengan akun admin.

1.3 Hasil akhir

Setelah kedua pertemuan diterapkan, halaman data produk memiliki kemampuan berikut:

1. **Ubah produk** melalui form yang sudah terisi nilai lama, dengan validasi kode unik yang mengecualikan produk yang sedang diedit.
2. **Hapus produk** dengan dialog konfirmasi; produk yang sudah memiliki riwayat transaksi tidak dihapus agar referensi historis tetap aman.
3. **Pencarian** berdasarkan nama atau kode produk.
4. **Filter kategori** yang bisa dikombinasikan dengan pencarian.
5. **Pagination** yang tetap membawa kata kunci dan filter saat berpindah halaman.



BAB 2 · PERTEMUAN 9

Mengubah dan Menghapus Data Produk

Route Model Binding, form edit, validasi update, dan konfirmasi hapus

2.1 Route Model Binding pada update dan delete

Sejak Pertemuan 7, rute resource `Route::resource('products', ProductController::class)` sudah mendaftarkan seluruh rute CRUD. Tiga rute yang kita pakai pada pertemuan ini adalah:

```
Route::middleware(['auth', 'role:admin'])->group(function () {
    Route::resource('categories', CategoryController::class);
    Route::resource('products', ProductController::class);
    Route::resource('users', UserController::class);
    Route::get('/reports/sales', [ReportController::class, 'sales'])->name('report.sales');
});
```

Method	URL	Method controller	Fungsi
GET	/products/{product}/edit	edit(Product $product)	Menampilkan form edit
PUT/PATCH	/products/{product}	update(UpdateProductRequest $request, Product $product)	Menyimpan perubahan
DELETE	/products/{product}	destroy(Product $product)	Menghapus produk

Nama parameter pada URL (`{product}`) harus sama dengan nama variabel di method controller (`$product`). Laravel otomatis mencari baris tabel `products` berdasarkan ID tersebut. Jika data tidak ditemukan, Laravel mengembalikan 404 tanpa kode `findOrFail()` manual.

URUTAN PARAMETER

Pada method `update()` , parameter `UpdateProductRequest $request` diletakkan sebelum `Product $product` . Laravel menyuntikkan Form Request lebih dulu, lalu model hasil binding. Urutan ini penting agar validasi berjalan sebelum data diubah.

2.2 UpdateProductRequest: validasi dan otorisasi

Form Request memisahkan aturan validasi dari controller. Aturan `unique:products,code` harus mengecualikan ID produk yang sedang diedit; jika tidak, menyimpan tanpa mengubah kode akan dianggap duplikat.

```
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class UpdateProductRequest extends FormRequest
{
    /**
     * Hanya admin yang boleh mengubah data produk.
     */
    public function authorize(): bool
    {
```

```
        return auth()->check() && auth()->user()->role === 'admin';
    }

    /**
     * Aturan validasi untuk memperbarui produk.
     * Unique code mengecualikan produk yang sedang diedit.
     *
     * @return array<string, \Illuminate\Contracts\Validation\ValidationRule|array<mixed>|string>
     */
    public function rules(): array
    {
        return [
            'category_id' => 'required|exists:categories,id',
            'code' => 'required|string|max:20|unique:products,code,' . $this->route('product')->id,
            'name' => 'required|string|max:150',
            'unit' => 'required|string|max:20',
            'price' => 'required|integer|min:0',
            'stock' => 'required|integer|min:0',
        ];
    }

    public function messages(): array
    {
        return [
            'code.unique' => 'Kode produk sudah digunakan, gunakan kode lain.',
        ];
    }
}
```

Penjelasan baris penting

- `authorize()` memastikan hanya admin yang boleh mengubah produk. Rute juga sudah dilindungi middleware `role:admin`, sehingga ada dua lapis pengaman.
- `unique:products,code,' . $this->route('product')->id` berarti "kode harus unik di tabel products, kecuali untuk baris dengan ID produk yang sedang diedit".
- `messages()` mengganti pesan bawaan agar lebih ramah pengguna.

> **KESALAHAN UMUM**
>
> Jika `authorize()` masih mengembalikan `false`, halaman update akan menghasilkan **403 Forbidden** meskipun rute dan form sudah benar. Pastikan method tersebut mengembalikan `true` atau memeriksa peran pengguna.

2.3 Method update()

Controller cukup memanggil `$request->validated()` agar hanya field yang lolos validasi yang masuk ke database. Setelah itu pengguna diarahkan kembali ke daftar produk dengan pesan sukses.

```
    public function update(UpdateProductRequest $request, Product $product)
    {
        $product->update($request->validated());

        return redirect()->route('products.index')->with('success', 'Produk berhasil diperbarui.');
    }
```

Pola `$product->update($request->validated())` aman karena model `Product` sudah memiliki `$fillable` untuk field yang diizinkan. Field di luar daftar tersebut tidak akan tersimpan meskipun dikirim oleh klien.

2.4 Form edit produk

Form edit harus mengisi nilai awal dari data produk, tetapi tetap memprioritaskan `old()` ketika validasi gagal. Perhatikan juga `@method('PUT')` karena HTML hanya mendukung GET dan POST.

```
<x-app-layout>
    <x-slot name="header">
        <div class="flex justify-between items-center">
            <h2 class="font-semibold text-xl text-gray-800">Ubah Produk</h2>
            <a href="{{ route('products.index') }}" class="text-sm text-gray-500">Kembali</a>
        </div>
    </x-slot>

    <div class="py-12">
        <div class="max-w-2xl mx-auto sm:px-6 lg:px-8">
            <x-card>
                <form action="{{ route('products.update', $product) }}" method="POST" class="space-y-4">
                    @csrf
                    @method('PUT')

                    <div>
                        <label class="block text-sm font-medium text-gray-700">Kategori</label>
                        <select name="category_id" class="mt-1 block w-full rounded-md border-gray-300">
                            <option value="" disabled>Pilih kategori</option>
                            @foreach ($categories as $category)
                                <option value="{{ $category->id }}"
                                    @selected(old('category_id', $product->category_id) == $category->id)>
                                    {{ $category->name }}
                                </option>
                            @endforeach
                        </select>
                        @error('category_id')
                            <p class="text-sm text-red-600 mt-1">{{ $message }}</p>
                        @enderror
                    </div>

                    <div>
                        <label class="block text-sm font-medium text-gray-700">Kode Produk</label>
                        <input type="text" name="code" value="{{ old('code', $product->code) }}"
                            class="mt-1 block w-full rounded-md border-gray-300">
                        @error('code')
                            <p class="text-sm text-red-600 mt-1">{{ $message }}</p>
                        @enderror
                    </div>

                    <div>
                        <label class="block text-sm font-medium text-gray-700">Nama Produk</label>
                        <input type="text" name="name" value="{{ old('name', $product->name) }}"
                            class="mt-1 block w-full rounded-md border-gray-300">
                        @error('name')
                            <p class="text-sm text-red-600 mt-1">{{ $message }}</p>
                        @enderror
                    </div>

                    <div class="grid grid-cols-3 gap-4">
                        <div>
                            <label class="block text-sm font-medium text-gray-700">Satuan</label>
                            <input type="text" name="unit" value="{{ old('unit', $product->unit) }}"
                                class="mt-1 block w-full rounded-md border-gray-300">
                            @error('unit')
                                <p class="text-sm text-red-600 mt-1">{{ $message }}</p>
                            @enderror
```

```
                    </div>

                    <div>
                        <label class="block text-sm font-medium text-gray-700">Harga</label>
                        <input type="number" name="price" value="{{ old('price', $product->price) }}"
                            class="mt-1 block w-full rounded-md border-gray-300">
                        @error('price')
                            <p class="text-sm text-red-600 mt-1">{{ $message }}</p>
                        @enderror
                    </div>

                    <div>
                        <label class="block text-sm font-medium text-gray-700">Stok</label>
                        <input type="number" name="stock" value="{{ old('stock', $product->stock) }}"
                            class="mt-1 block w-full rounded-md border-gray-300">
                        @error('stock')
                            <p class="text-sm text-red-600 mt-1">{{ $message }}</p>
                        @enderror
                    </div>
                </div>

                <div class="flex items-center gap-3">
                    <button type="submit" class="bg-indigo-600 text-white px-4 py-2 rounded-md">
                        Simpan Perubahan
                    </button>
                    <a href="{{ route('products.index') }}" class="text-gray-500 px-4 py-2">Batal</a>
                </div>
            </form>
        </x-card>
    </div>
</div>
</x-app-layout>
```

Hal yang wajib ada di form edit

- `@csrf` untuk token keamanan.
- `@method('PUT')` agar Laravel memanggil method `update()`, bukan `store()`.
- `old('field', $product->field)` agar input pengguna tidak hilang saat validasi gagal.
- `@error('field') ... @enderror` untuk menampilkan pesan validasi tepat di bawah field terkait.
- `@selected(...)` pada dropdown kategori agar pilihan lama tetap terpilih.

2.5 Method destroy() dan konfirmasi hapus

Produk yang sudah pernah terjual memiliki baris di `transaction_details`. Foreign key pada tabel tersebut tidak memakai *cascade delete*, sehingga penghapusan produk akan gagal. Controller menangkap `QueryException` dan menampilkan pesan yang jelas alih-alih halaman error 500.

```
    public function destroy(Product $product)
    {
        try {
            $product->delete();
        } catch (QueryException $e) {
            return redirect()->route('products.index')
                ->with('error', 'Produk tidak dapat dihapus karena sudah memiliki riwayat transaksi.');
        }

        return redirect()->route('products.index')->with('success', 'Produk berhasil dihapus.');
    }
```

Di sisi tampilan, tombol hapus dibungkus form kecil dengan konfirmasi JavaScript. Atribut `data-name` dipakai agar nama produk yang mengandung tanda kutip tetap aman ketika dimasukkan ke dialog konfirmasi.

```
<form action="{{ route('products.destroy', $product) }}" method="POST"
    data-name="{{ $product->name }}"
    onsubmit="return confirm('Yakin ingin menghapus produk ' + this.dataset.name + '?')"
    class="inline">
    @csrf
    @method('DELETE')
    <button type="submit" class="text-red-600">Hapus</button>
</form>
```

PERTIMBANGAN SOFT DELETE

Untuk data yang punya nilai historis, penghapusan permanen sering tidak tepat. Alternatifnya gunakan *soft delete*: data tidak benar-benar hilang, hanya ditandai `deleted_at`. Laporan lama tetap bisa menampilkan nama produk melalui `withTrashed()`.

```
use Illuminate\Database\Eloquent\SoftDeletes;

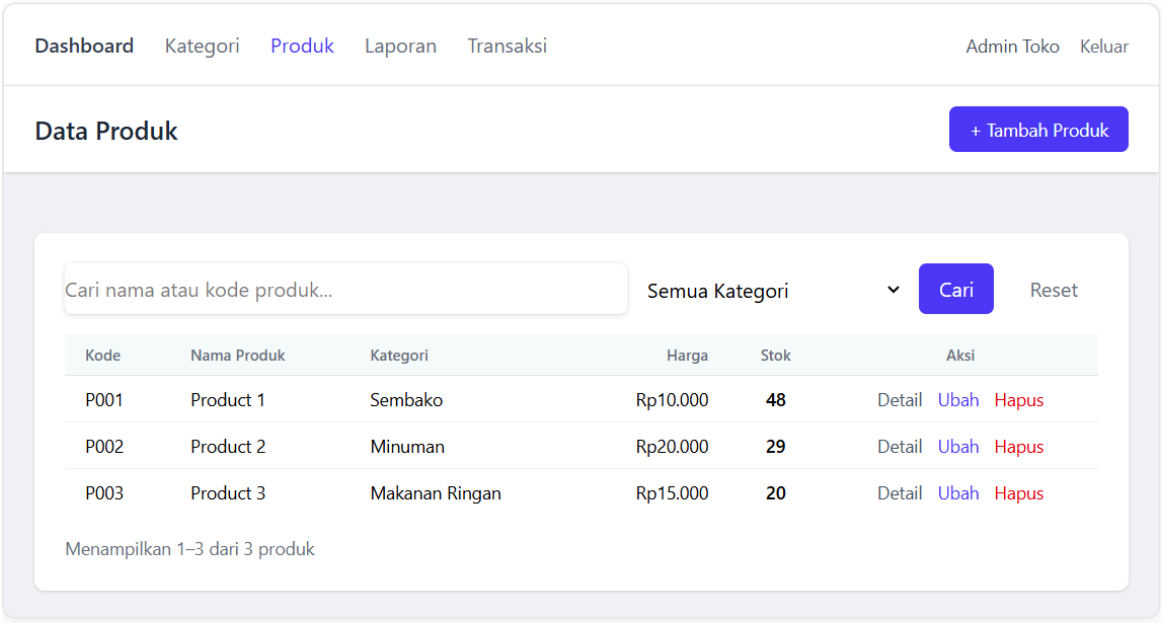
class Product extends Model
{
    use SoftDeletes;
}

// Pada migrasi:
$table->softDeletes();

// Mengambil produk yang sudah dihapus (untuk laporan):
Product::withTrashed()->find($id);
```

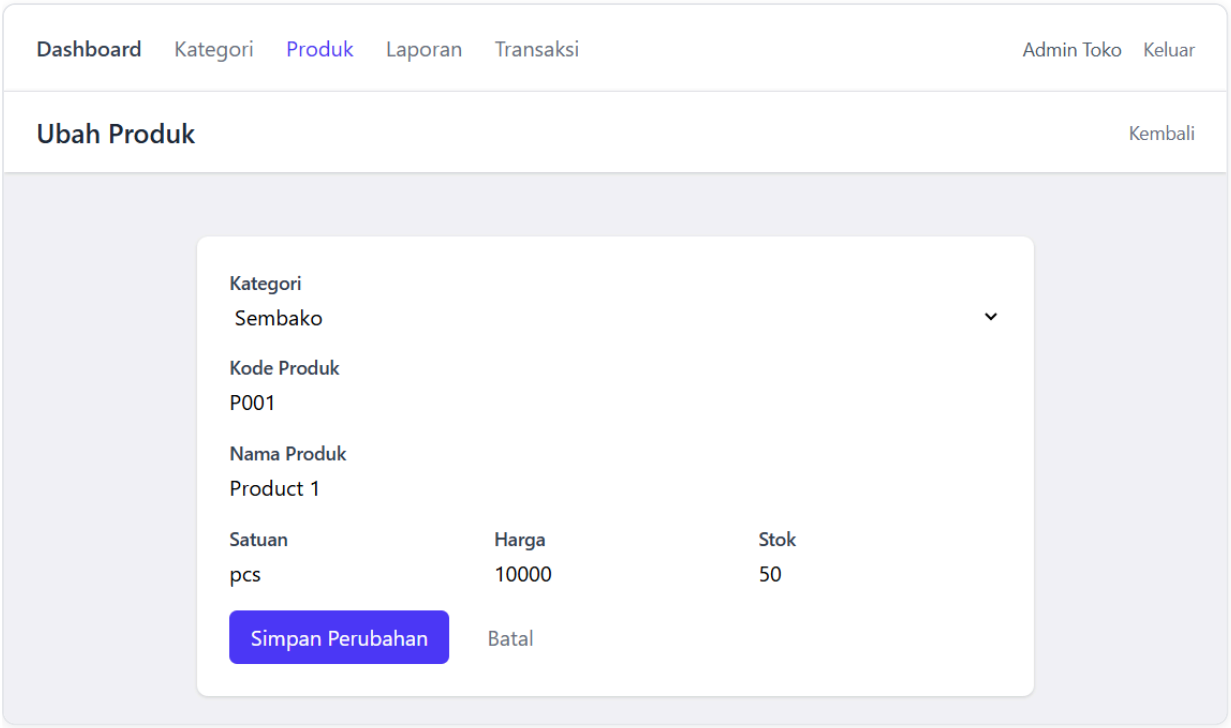
2.6 Menguji fitur ubah dan hapus

Buka menu **Produk**. Setiap baris memiliki tombol **Detail**, **Ubah**, dan **Hapus**.



Gambar 2.1 — Daftar produk dengan form pencarian, filter kategori, dan tombol aksi.

Klik **Ubah** pada salah satu produk. Form akan terisi data yang ada, termasuk kategori, kode, nama, satuan, harga, dan stok.



Gambar 2.2 — Form ubah produk dengan nilai awal dari database.

Hasil pengujian

Skenario	Hasil yang diamati	Status
Ubah nama/harga produk	Pesan “Produk berhasil diperbarui.”, data berubah	Berhasil
Ubah kode menjadi kode produk lain	Pesan “Kode produk sudah digunakan, gunakan kode lain.”	Berhasil
Hapus produk tanpa riwayat transaksi	Pesan “Produk berhasil dihapus.”, produk hilang	Berhasil
Hapus produk yang punya riwayat transaksi	Pesan produk tidak dapat dihapus, data tetap ada	Berhasil
Klik Batal pada dialog konfirmasi	Tidak ada perubahan data	Berhasil

CATATAN PENGUJIAN

Produk uji yang dibuat khusus untuk pengujian dihapus kembali setelah tes, sehingga daftar produk akhir tetap berisi data asli. Produk yang sudah memiliki riwayat transaksi sengaja dipertahankan.

BAB 3 · PERTEMUAN 10

Search, Filter Kategori, dan Pagination

Query parameter, pencarian nama/kode, filter kategori, dan pagination yang konsisten

3.1 Mengenal query parameter

Query parameter adalah bagian URL setelah tanda tanya (`?`) yang berisi pasangan *key=value*. Beberapa parameter dipisahkan dengan `&`. Contoh:

```
/products?search=minyak&category_id=1
```

Di server, nilai tersebut dapat dibaca dengan `$request->query('search')`, `$request->get('search')`, atau `request('search')`. Query parameter cocok untuk pencarian karena hasilnya bisa di-bookmark dan dibagikan sebagai tautan — berbeda dari pencarian berbasis session yang nilainya tidak ikut tersimpan di URL.

MENGAPA BUKAN SESSION?

Jika kata kunci disimpan di session, dua tab browser bisa saling menimpa dan tautan yang dibagikan tidak menampilkan hasil yang sama. Query parameter membuat state pencarian eksplisit di URL.

3.2 Method index(): pencarian, filter, dan pagination

Method `index()` membangun query secara bertahap. Pencarian dibungkus `where(function ($q) ...)` agar kondisi `OR` untuk nama dan kode tidak mengganggu filter kategori.

```php
public function index(Request $request)
{
    $query = Product::with('category');

    if ($request->filled('search')) {
        $keyword = $request->search;

        $query->where(function ($q) use ($keyword) {
            $q->where('name', 'like', "%{$keyword}%")
                ->orWhere('code', 'like', "%{$keyword}%");
        });
    }

    if ($request->filled('category_id')) {
        $query->where('category_id', $request->category_id);
    }

    $products = $query->latest()->paginate(10)->withQueryString();
    $categories = Category::orderBy('name')->get();

    return view('master-data.product.index', compact('products', 'categories'));
}
```

Penjelasan baris penting

- `Product::with('category')` memuat relasi kategori sekaligus untuk menghindari *N+1 query* saat tabel merender nama kategori.
- `$request->filled('search')` memastikan parameter ada dan tidak kosong. Jika kosong, kondisi pencarian tidak ditambahkan.

- Pencarian memakai `LIKE %keyword%` pada kolom `name` dan `code`.
- `$request->filled('category_id')` menambahkan filter `where('category_id', ...)`.
- `paginate(10)` membatasi 10 baris per halaman, sedangkan `withQueryString()` menambahkan kembali `search` dan `category_id` ke tautan pagination.
- `$categories` dikirim ke view untuk mengisi dropdown filter.

3.3 Form pencarian dan filter kategori

Form menggunakan method `GET` agar parameter muncul di URL. Nilai input diisi kembali dengan `request('search')` dan `request('category_id')`, sehingga pengguna melihat filter yang sedang aktif.

```
                <form method="GET" action="{{ route('products.index') }}" class="flex flex-wrap gap-3 mb-4">
                    <input type="text" name="search" value="{{ request('search') }}"
                        placeholder="Cari nama atau kode produk..."
                        class="flex-1 min-w-[220px] rounded-md border-gray-300 shadow-sm">

                    <select name="category_id" class="rounded-md border-gray-300">
                        <option value="">Semua Kategori</option>
                        @foreach ($categories as $category)
                            <option value="{{ $category->id }}" @selected(request('category_id') ==
$category->id)>

                                {{ $category->name }}
                            </option>
                        @endforeach
                    </select>

                    <button type="submit" class="bg-indigo-600 text-white px-4 py-2 rounded-md">Cari</button>
                    <a href="{{ route('products.index') }}" class="px-4 py-2 text-gray-500">Reset</a>
                </form>
```

@SELECTED

`@selected(request('category_id') == $category->id)` menghasilkan atribut `selected` pada opsi yang cocok. Ini membuat dropdown tetap menunjukkan kategori yang sedang difilter setelah halaman dimuat ulang.

3.4 Pagination yang mempertahankan filter

Di bawah tabel, tampilkan jumlah data dan tautan halaman. Bagian pentingnya adalah `{{ $products->links() }}`; karena query sudah memakai `withQueryString()`, tautan halaman kedua otomatis membawa `?search=...&category_id=...`.

```
<div class="mt-4 flex items-center justify-between">
    <p class="text-sm text-gray-500">
        Menampilkan {{ $products->firstItem() }}&ndash;{{ $products->lastItem() }}
        dari {{ $products->total() }} produk
    </p>

    {{ $products->links() }}
</div>
```

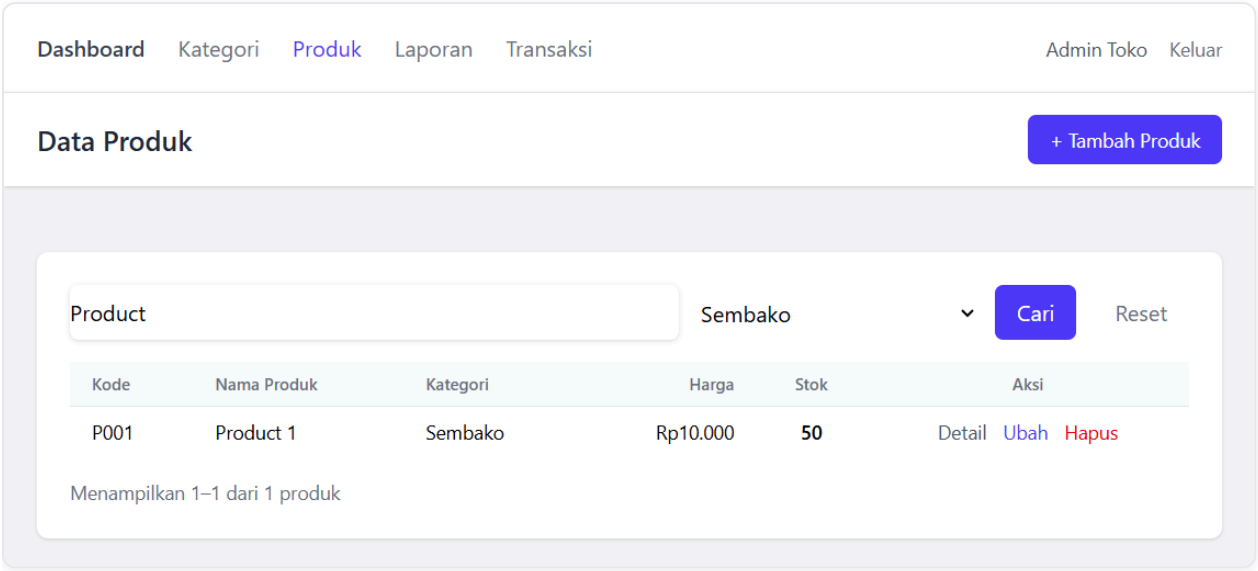
Perilaku yang diharapkan

- Halaman 1 menampilkan 10 data pertama; halaman 2 menampilkan sisanya.
- Saat berpindah halaman, kata kunci dan kategori tetap aktif.

- Jika hasil pencarian kosong, tabel menampilkan pesan “Belum ada data produk.” dan pagination tidak menampilkan halaman.
- Tombol **Reset** mengembalikan daftar ke kondisi tanpa filter.

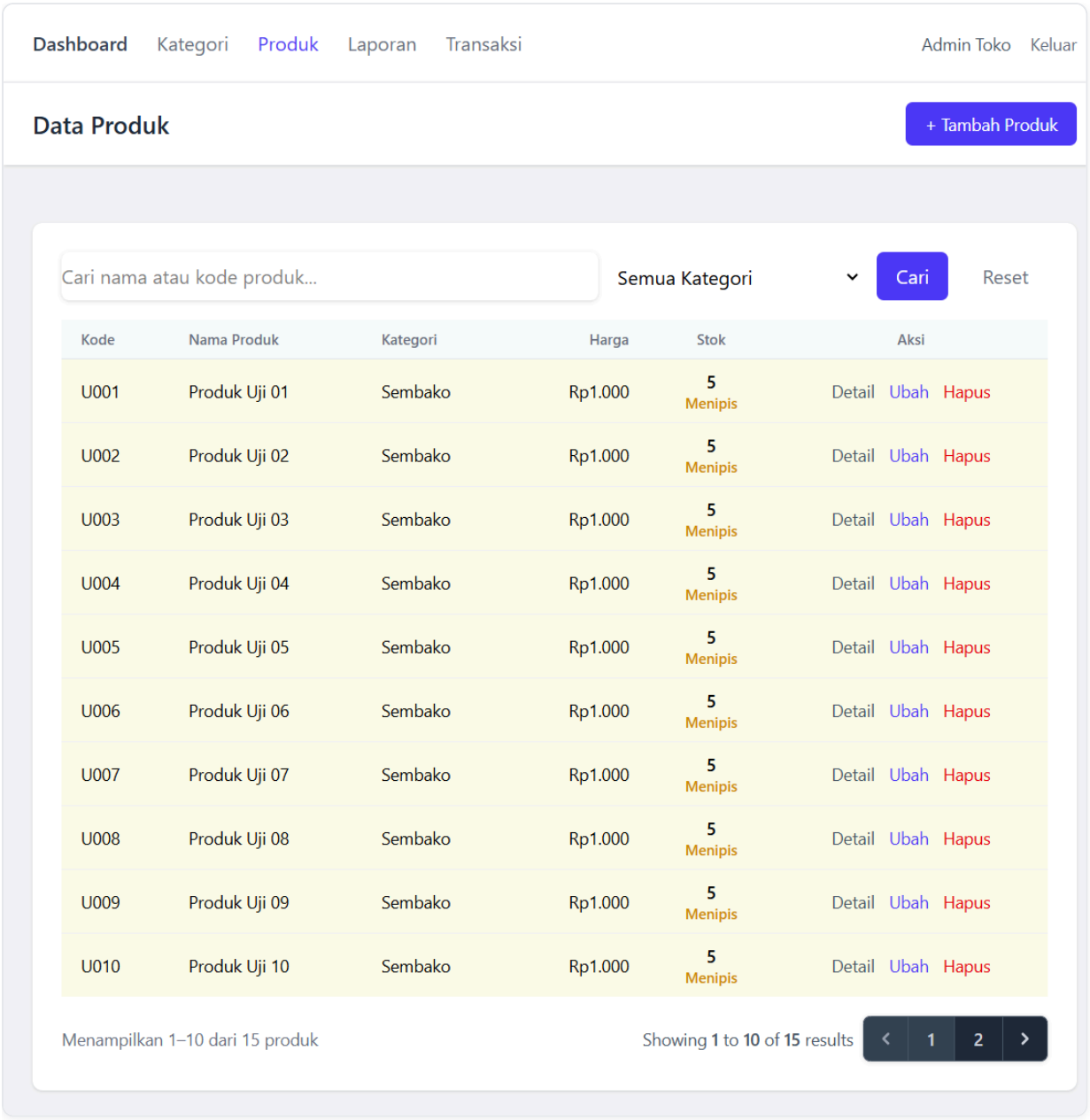
3.5 Menguji pencarian, filter, dan pagination

Masukkan kata kunci **Product** dan pilih kategori **Sembako**, lalu klik **Cari**. Hasilnya hanya produk yang cocok dengan kedua kondisi.



Gambar 3.1 — Hasil pencarian nama produk sekaligus filter kategori.

Untuk melihat pagination, daftar diisi 15 produk (12 data uji ditambahkan sementara). Pagination membagi data menjadi dua halaman dengan 10 baris per halaman.



Gambar 3.2 — Pagination pada daftar produk; data uji ditambahkan sementara untuk memperlihatkan kontrol halaman.

Hasil pengujian

Skenario	Hasil yang diamati	Status
Cari berdasarkan nama “Product”	Menampilkan produk yang namanya mengandung kata tersebut	Berhasil
Cari berdasarkan kode “P001”	Menampilkan Product 1	Berhasil
Filter kategori Sembako	Hanya produk kategori Sembako yang tampil	Berhasil
Gabungan pencarian + filter	Hasil memenuhi kedua kondisi	Berhasil
Klik Reset	Kembali ke daftar lengkap tanpa filter	Berhasil
Pindah ke halaman 2	Menampilkan sisa data, filter tetap terbawa	Berhasil
Pencarian tanpa hasil	Tabel kosong dengan pesan yang jelas	Berhasil

3.6 Catatan performa

- Pola `LIKE '%kata%'` tidak memanfaatkan index secara optimal karena wildcard di awal. Untuk data besar, pertimbangkan full-text search atau index khusus.
- `with('category')` penting agar tabel tidak melakukan satu query kategori untuk setiap baris produk.
- Batasi jumlah baris per halaman (misalnya 10–25) agar halaman tetap ringan.
- Validasi `category_id` di sisi server jika filter dipakai untuk query yang lebih kompleks; saat ini nilai hanya dipakai pada `where` sehingga aman dari SQL injection.

BAB 4

Kesimpulan dan Checklist

4.1 Kesimpulan

Pertemuan 9 melengkapi operasi CRUD data produk. Route Model Binding membuat method `edit()`, `update()`, dan `destroy()` lebih ringkas karena Laravel mencarikan model berdasarkan parameter URL. Form edit yang baik memadukan `@csrf`, `@method('PUT')`, `old()`, dan pesan validasi per field. Penghapusan data perlu mempertimbangkan relasi historis; produk yang sudah terjual sebaiknya tidak dihapus permanen.

Pertemuan 10 menambahkan kemampuan penelusuran data. Query parameter membuat state pencarian eksplisit di URL, sedangkan `withQueryString()` menjaga filter tetap aktif saat berpindah halaman. Kombinasi pencarian, filter kategori, dan pagination membuat daftar produk tetap nyaman dipakai meskipun jumlah data bertambah.

4.2 Checklist implementasi

- ✓ Rute resource produk tersedia dan dilindungi middleware admin.
- ✓ `UpdateProductRequest` mengizinkan admin dan memakai unique dengan pengecualian ID.
- ✓ Form edit memakai `@csrf`, `@method('PUT')`, `old()`, dan `@error`.
- ✓ Tombol hapus memakai konfirmasi dan method `DELETE`.
- ✓ Produk dengan riwayat transaksi tidak terhapus dan menampilkan pesan yang jelas.
- ✓ Pencarian memeriksa kolom nama dan kode.
- ✓ Filter kategori bisa digabung dengan pencarian.
- ✓ Pagination memakai `paginate(10)` dan `withQueryString()`.
- ✓ Form filter mengisi ulang nilai dengan `request()`.
- ✓ Relasi kategori dimuat dengan `with('category')`.

BAB 5

Pemecahan Masalah

5.1 Error 403 saat menyimpan perubahan

Periksa `authorize()` pada `UpdateProductRequest`. Jika masih `return false;`, Laravel menolak request sebelum method `update()` dijalankan. Pastikan juga pengguna memiliki peran admin.

5.2 Validasi kode unik gagal padahal kode tidak berubah

Aturan unique harus mengecualikan ID produk yang sedang diedit: `unique:products,code,' . $this->route('product')->id`. Tanpa pengecualian, kode milik produk itu sendiri dianggap duplikat.

5.3 Form edit tidak menyimpan (method tidak diizinkan)

HTML hanya mendukung GET dan POST. Tambahkan `@method('PUT')` di dalam form dan pastikan rute `PUT/PATCH /products/{product}` terdaftar. Cek juga `action="{{ route('products.update', $product) }}"`.

5.4 Gagal menghapus produk karena foreign key

Produk yang sudah dipakai di `transaction_details` tidak bisa dihapus jika foreign key tidak memakai cascade. Ini disengaja agar riwayat transaksi tetap utuh. Gunakan soft delete atau nonaktifkan produk alih-alih menghapus permanen.

5.5 Pencarian tidak mengembalikan hasil

- Pastikan input memiliki atribut `name="search"`.
- Pastikan form memakai `method="GET"`.
- Periksa kondisi `$request->filled('search')`.
- Untuk pencarian kode, pastikan kolom yang diperiksa memang `code`, bukan `name` saja.

5.6 Pagination menghilangkan filter

Tambahkan `->withQueryString()` setelah `paginate()`. Tanpa method itu, tautan halaman hanya membawa parameter `page`.

5.7 Error 419 Page Expired

Token CSRF hilang atau kedaluwarsa. Pastikan form memiliki `@csrf` dan session driver berjalan normal. Jika baru mengubah konfigurasi, jalankan `php artisan optimize:clear`.

BAB 6

Daftar Berkas

Seluruh berkas berikut berada di dalam folder proyek `C:\xampp\htdocs\FPW\FPW_Test1`.

No	Berkas	Status	Keterangan
1	app/Http/Controllers/ProductController.php	Diubah	index (search/filter/pagination), edit, update, destroy
2	app/Http/Requests/UpdateProductRequest.php	Diubah	Authorize admin + unique pengecualian
3	resources/views/master-data/product/index.blade.php	Diubah	Form pencarian, filter kategori, aksi, pagination
4	resources/views/master-data/product/edit.blade.php	Diubah	Form edit lengkap dengan old() dan error
5	routes/web.php	Diubah	Perapian rute resource produk

CATATAN

Berkas `show.blade.php` dan folder `resources/views/pos/` merupakan bagian dari Pertemuan 11–12, sehingga tidak dibahas dalam laporan ini.

BAB 7

Referensi

1. Laravel 13 Documentation — Routing: Route Model Binding dan Resource Routes.
2. Laravel 13 Documentation — Validation: Form Request, unique rule, dan error messages.
3. Laravel 13 Documentation — Eloquent: Mass Assignment dan Soft Deletes.
4. Laravel 13 Documentation — Database: Query Builder (where, orWhere, LIKE).
5. Laravel 13 Documentation — Pagination: withQueryString dan custom views.
6. Modul Praktikum FPW Pertemuan 9 — Mengubah dan Menghapus Data Produk.
7. Modul Praktikum FPW Pertemuan 10 — Search, Filter Kategori, dan Pagination.

PENUTUP

Dengan selesainya Pertemuan 9 dan 10, data master produk sudah bisa diubah, dihapus secara aman, dicari, difilter, dan ditampilkan per halaman. Kemampuan ini menjadi fondasi sebelum masuk ke halaman detail produk serta transaksi Point of Sale pada pertemuan berikutnya.